DSA Unit 1 - MCQ Practice Set

Questions only - Answer key at the end

Compiled from the three uploaded PESU Academy quiz PDFs. Original question wording and options are retained; website navigation/footer text is removed.

Section A - L4 Revision (C Fundamentals) - 30 questions

Source: dsa unit 1,1.pdf

Section B - L8 Stack / List - 30 questions

Source: dsa unit 1,2.pdf

Section C - L13 Skiplist / Multilist - 40 questions

Source: dsa unit 1,3.pdf

Section A - L4 Revision (C Fundamentals)

Source: dsa unit 1,1.pdf

1) int *p = malloc(10 * sizeof(int)); printf("%d", sizeof(p)); What will be printed?

A) sizeof an int
B) sizeof a pointer
C) size of an array of 10 integer
D) 10

2) What is the difference between int *p[5]; and int (*p)[5]?

A) They are identical
B) int *p[5]; is an array of 5 pointers to int; int (*p)[5]; is a pointer to an array of 5 ints
C) int *p[5]; is a pointer to an array of 5 ints; int (*p)[5]; is an array of 5 pointers
D) Both are syntax errors

3) What will happen if you increment a pointer to a structure?

A) It moves by 1 byte
B) It moves by the size of the structure
C) It moves by the size of an integer
D) Compilation error

4) What will be the output?

```
void fun(int *p){ int x = 20; p = &x; }
int main(){ int x = 10; fun(&x); printf("%d", x); }
```

A) 10
B) 20
C) Garbage
D) Error

5) Consider the following C program. What is the result of the pointer subtraction?

```
#include <stdio.h>
#include <stddef.h>

int main() {
   int a[3] = {1, 2, 3};
   int b[3] = {4, 5, 6};
   ptrdiff_t diff = &a[2] - &b[0];
   return 0;
}
```

A) Undefined behavior
B) Implementation-defined fixed offset
C) -3
D) 3

6) Assume integer takes 4 bytes of memory and this is a 64-bit machine. What is the output?

```
int main()
{
   int a[5] = {1,2,3}, *p = a;
   printf("%d and %d", sizeof(a), sizeof(p));
}
```

A) 12 and 12
B) 20 and 8
C) 20 and 20
D) None of the above

7) Consider: int f(int x) { if (x == 0) return 0; return x + f(x - 1); } What value is returned by f(3)?

A) 3
B) 5
C) 6
D) 9

8) Consider: int a[] = {5, 10, 15, 20}; int *p = a; p = p + 2; Which expression gives the value 15?

A) *p
B) *(p + 1)
C) *(p - 1)
D) *a + 2

9) void fun(){ int a[] = {5, 10, 15, 20, 35}; int *p = &a[1]; int *q = &a[4]; printf("%ld", q - p); } What is the output of function fun()?

A) 3
B) 25
C) 15
D) Undefined behaviour

10) What does sizeof(arr) evaluate to inside the function func?

```
#include <stdio.h>
void func(int arr[10]) {
   printf("%zu", sizeof(arr));
}
```

A) 1
B) 10
C) 10*sizeof(int)
D) sizeof(int*)

11) What happens when a variable is passed to a function by value?

A) The original variable is modified directly
B) A copy of the variable's value is passed, so changes inside the function do not affect the original
C) The function cannot use the variable at all
D) It always causes a compilation error

12) A programmer wants a function to modify the contents of a structure variable declared in the calling function. Which of the following statements is most accurate?

A) Passing the structure directly to the function guarantees that any modifications made inside the function will be reflected in the caller because structures are composite data types.
B) Passing a structure directly to a function creates a copy of the structure; therefore, modifications inside the function affect only the copy unless the structure's address is passed.
C) Passing a structure directly to a function is illegal in C because only pointers to structures can be passed as arguments.
D) Passing a structure directly to a function causes the compiler to automatically convert the structure into a pointer to avoid copying overhead.

13) Which of the following statements is true about the return type of functions in C?

A) Functions can return any type, including arrays and functions
B) Functions can return any type except arrays and functions
C) Functions cannot return structures
D) The default return type of a function is void

14) What will be the output?

```
struct Test { int x; };
void fun(struct Test t) { t.x = 50; }
int main() { struct Test t = {20}; fun(t); printf("%d", t.x); }
```

A) 20
B) 50
C) Garbage
D) Compilation Error

15) What is the 'base case' in a recursive function?

A) The first call made to the function
B) The condition that stops further recursive calls
C) A syntax error in recursion
D) The loop counter used in recursion

16) struct what { int a; int *b; }; struct what x; int z = 200; x.a = 100; x.b = &z; z = 300; What is the value of *x.b?

A) 100
B) 200
C) 300
D) undefined

17) Consider the following code. What is true after free(p)?

```
int *p = malloc(3 * sizeof(int));
p[0] = 10; p[1] = 20; p[2] = 30;
int *q = p + 1;
free(p);
```

A) Before free(p), both p and q point to the first element of the allocated block; after free(p), only p becomes invalid.
B) Before free(p), p points to p[0] and q points to p[1]; after free(p), both p and q are dangling pointers.
C) Before free(p), q points to a separate memory block; after free(p), q remains valid because only the memory pointed to by p is released.
D) free(p) releases only the first integer, so q continues to point to a valid allocated integer containing 20.

18) int fun(int n){ if (n == 1) return 1; return n + fun(n / 2); } What is the value of fun(8)?

A) 11
B) 14
C) 15
D) 16

19) What will be the output of the given code?

```
void fun(int a[]){ a++; printf("%d", a[0]); }
int main(){ int arr[] = {5,10,15}; fun(arr); }
```

A) 5
B) 10
C) 15
D) Compilation Error

20) int z = 10; struct what { int a; int *b; }; struct what x = {20, &z}; struct what y = x; *x.b = 30; What is the value of *y.b?

A) 10
B) 20
C) 30
D) Compilation Error

21) Consider the following code. What is the output?

```
int a[] = {10, 20, 30, 40};
int *p = a;
printf("%d %d %d\n", *p, *(p + 2), p[3]);
```

A) 10 32 30
B) 10 30 40
C) 10 30 30
D) 10 40 30

22) Consider the following C program. What happens when *p is evaluated after f returns?

```
#include <stdio.h>
#include <stdlib.h>
int *f(){ int x=10; return &x; }
int main(){ int *p; p=f(); printf("\n %d", *p); }
```

A) 10
B) Syntax Error
C) Dangling pointer
D) None of the above

23) What is the correct declaration for a pointer to a function that takes an int and returns a float?

A) float *fp(int);
B) float (*fp)(int);
C) (float*) fp(int);
D) float fp(*int);

24) Consider the following code. What is the output?

```
#include <stdio.h>
void modify(int *p){ *(p + 1) = *(p + 1) + *p; }
int main(){ int a[] = {5, 10, 15}; modify(a); printf("%d %d %d", a[0], a[1], a[2]); return 0; }
```

A) 5 10 15
B) 5 15 15
C) 5 20 15
D) 10 15 20

25) Besides the sum of individual member sizes, what affects the total size of a structure?

A) The number of functions in the program
B) Structure padding and memory alignment
C) The number of pointers to the structure created
D) The number of variables of structure created

26) int a[] = {1, 2, 3}; int *p = a; What happens if we do p == a?

A) Syntax error
B) 1
C) 0
D) runtime error

27) What will be the output of this program?

```
#include<stdio.h>
#include<stdlib.h>
int main() { int *p, *q; p = (int *) malloc(sizeof(int)); q = p; *p = 10; free(q); printf("\n %d", *q); }
```

A) Undefined behavior as q is a dangling pointer.
B) 10
C) some address
D) Null

28) What is the output of the following program?

```
int main()
{
   int a[5] = {1,2,3}, x, y;
   x = (*a)++;
   y = ++*a;
   printf("\n x=%d and y=%d", x, y);
}
```

A) A. x = 1 and y = 2
B) A. x = 1 and y = 1
C) A. x = 1 and y = 3
D) A. x = 2 and y = 3

29) Which statement correctly changes the number of pages to 300?

```
struct Book { int pages; float price; };
struct Book b = {250, 450.50};
```

A) Book.pages = 300;
B) b->pages = 300;
C) b.pages = 300;
D) *b.pages = 300;

30) What happens if fewer values are provided during array initialization? int a[5] = {1, 2};

A) Compilation error
B) Remaining elements are initialized to 0
C) Remaining elements contain garbage values
D) Array size becomes 2

Section B - L8 Stack / List

Source: dsa unit 1,2.pdf

1) Two stacks, S1 and S2, are initially empty. Perform the following operations. What is the final content of S2 from TOP to BOTTOM?

```
S1.push(10)
S1.push(20)
S1.push(30)
S2.push(S1.pop())
S2.push(S1.pop())
S1.push(40)
S2.push(S1.pop())
```

A) 10, 20, 30
B) 20, 30, 40
C) 40, 20, 30
D) 30, 20, 40

2) Consider the following stack operations. What are the values of x, y, and z?

```
push(5);
push(10);
push(15);
int x = pop();
if (x > 10)
   push(x + 5);
else
   push(x - 5);
push(20);
int y = pop();
int z = pop();
```

A) x = 15, y = 20, z = 20
B) x = 15, y = 20, z = 10
C) x = 10, y = 20, z = 15
D) x = 15, y = 10, z = 20

3) List with a Header Node: A singly linked list uses a header (dummy) node that does not contain a user element. What is the major advantage of this design?

A) It eliminates the need for pointers between nodes.
B) It guarantees that insertion and deletion require a few operations independent of the number of elements in the list.
C) It simplifies boundary cases such as insertion/deletion at the first actual element.
D) It allows the list to contain unlimited elements.

4) We separate the interface from the implementation. Why should the user not access the implementation?

A) This is for security reason
B) This is to make the user's life easier
C) The implementation may be changed; then the user will be affected
D) This is just a convention

5) Consider: struct node *p = head; // there is a header node. If initially head points to the header node and the list is HEADER -> 10 -> 20 -> 30 -> NULL, where does p point after execution?

```
p = p->next;
p = p->next;
```

A) HEADER
B) 10
C) 20
D) 30

6) Given the following recursive function, what does the above function do?

```
struct node {
   int key_;
   struct node* next_;
};
typedef struct node node_t;

// pres points to the first node in a list if the list is not empty;
// otherwise it is NULL.
void what(node_t* pres)
{
   if(pres == NULL)
      return 0;
   else
      return 2 + what(pres->next_->next_);
}
```

A) gives the correct count of number of nodes in the list
B) gives the correct count if the number of nodes is even, else one less
C) may end up dereferencing a dangling pointer
D) none of these

7) If top_ indicates the index in the array where an element can be pushed, which of the following is the right code for push? Assume that the stack is not full.

```
struct stack
{
   char ch_[100];
   int top_;
};
typedef struct stack stack_t;
```

A) void push(stack_t *ptr_stack, char ch) { ptr_stack->ch_[++ptr_stack->top_] = ch; }
B) void push(stack_t *ptr_stack, char ch) { ptr_stack->ch_[ptr_stack->top_++] = ch; }
C) void push(stack_t *ptr_stack, char ch) { ptr_stack->ch_[ptr_stack->top_] = ch; }
D) void push(stack_t *ptr_stack, char ch) { ptr_stack->ch_[ptr_stack->++top_] = ch; }

8) Consider the following list-freeing function. Why must r = q->next_ be executed before free(q)?

```
void freelist(node_t *q)
{
   node_t *r;
   while(q != NULL)
   {
      r = q->next_;
      free(q);
      q = r;
   }
}
```

A) free() changes the value of q to NULL
B) After free(q), accessing q->next_ is invalid
C) q->next_ can only be accessed through r
D) free() automatically frees the next node

9) What is the postfix expression for the following: a - b + c

A) a - b - c
B) a b c - +
C) a b + c -
D) a b - c +

10) p = head; while(p->next != NULL && p->next->data < item) p = p->next; What is the purpose of this loop?

A) Find the last node
B) Find the position for insertion
C) Delete a node
D) Reverse the list

11) Suppose a List ADT is implemented using an array in one program and a linked list in another program. Which statement is correct?

A) They are two different ADTs
B) Only the array implementation is valid
C) Both can implement the same List ADT
D) A List ADT cannot have multiple implementations

12) Which statement best describes a List as an Abstract Data Type (ADT)?

A) It specifies that elements must be stored in contiguous memory locations.
B) It defines the operations and their behavior without specifying the implementation.
C) It requires every element to have a unique key.
D) It can only be implemented using arrays.

13) Evaluate this postfix expression: 8 2 / 3 - 4 2 * +

A) 9
B) 11
C) 13
D) 15

14) Consider a singly linked list with a header node. The header node does not contain an actual list element. The structure is HEADER -> 10 -> 20 -> 30 -> NULL. Which statement is correct?

A) The header node must always contain the first data element.
B) The header node simplifies insertion and deletion at the beginning of the list.
C) The header node increases the number of data elements by one.
D) The header node must always be the last node.

15) Consider: 10 -> 20 -> 30 -> 40 -> NULL. What is the resulting list reachable from head?

```
struct node *p = head;
struct node *q = p->next->next;
p->next = q;
```

A) 10 -> 20 -> 30 -> 40
B) 10 -> 30 -> 40
C) 20 -> 30 -> 40
D) 10 -> 40

16) Consider: A * (B + C) / D. The postfix expression is:

A) ABC+*D/
B) AB+C*D/
C) ABC*+D/
D) A*BC+D/

17) Consider the following representation. After inserting 10, 20, 30 at the beginning, the list is head -> [HEADER] -> [10] -> [20] -> [30] -> NULL. Which statement is always correct, including when the list is empty?

A) head == NULL indicates an empty list
B) head->next == NULL indicates an empty list

C) head->next->next == NULL indicates an empty list
D) head->data == -1 indicates an empty list

18) Suppose the input sequence is 1 2 3 and a stack is used. Either the element can be pushed or output directly. At the end, all the elements in the stack are popped out. Which of the following cannot be obtained as the output sequence?

A) 1 2 3
B) 2 1 3
C) 3 2 1
D) 3 1 2

19) Consider a Stack ADT supporting push(x), pop(), and peek(). Two implementations are provided: one using an array and another using a linked list. Which statement is most accurate?

A) Programs using the Stack ADT must be rewritten whenever the implementation changes.
B) The client program should depend on the internal representation to achieve better performance.
C) Both implementations are valid if they provide the same externally observable stack behavior, even though their internal representations differ.
D) A Stack ADT cannot be implemented using different data structures because its operations are implementation-dependent.

20) In an ascending ordered linked list having n nodes, while searching for key X,

A) the number of comparisons will be always n
B) the number of comparisons will always be less than n
C) the comparison could stop as soon as the key of the node in the list is more than X
D) the comparison could stop as soon as the key of the node in the list is less than X

21) If a header node is used, an empty linked list can be represented as:

A) head == NULL
B) head->next == NULL
C) head->next->next == NULL
D) head == tail

22) This is a function to check whether a given list is ordered based on key_ in ascending order. Fill up the blank.

```
int is_ordered(list_t *ptr_list)
{
   node_t* pres = ptr_list->head_;
   int is_valid = 1;
   while(is_valid && pres != NULL && pres->next_ != NULL)
   {
      is_valid = _____ ;
      pres = pres->next_;
   }
   return is_valid;
}
```

A) pres->key_ <= pres->next_->key_
B) pres->key_ > pres->next_->key_
C) pres->next_->key_ <= pres->next_->next_->key_
D) pres->next_->key_ > pres->next_->next_->key_

23) An ordered list contains: 10 -> 20 -> 35 -> 50 -> 80. A new element 40 must be inserted while preserving the ordering. Which sequence of operations is logically necessary in a linked implementation?

A) Insert at the beginning, then sort the entire list.
B) Traverse until the first node containing a value greater than 40, then insert before it.

C) Insert after 80, then move the node backward until it reaches its position.
D) Replace 35 with 40 and append 35 at the end.

24) Which of the following best describes an Abstract Data Type (ADT)?

A) A specific memory layout used to store data
B) A programming language construct used to define variables
C) A list of what operations can be performed
D) A list of how operations can be performed

25) A list with a header node is represented as HEADER -> 20 -> 30 -> 40 -> NULL. To insert 10 at the beginning, which code is correct?

A) new->next = header; header = new;
B) new->next = header->next; header->next = new;
C) header->next = new; new->next = NULL;
D) new = header->next; header->next = new;

26) Consider an ordered linked list: 20 -> 30 -> 40 -> 50. A new node containing 10 has to be inserted. Which pointer must be updated?

A) Only head
B) Only head->next
C) Only the last node
D) No pointer

27) Suppose a Stack ADT is implemented using an array. Which of the following is false? If the implementation is changed to a linked list,

A) the client should modify his code
B) the client might have to recompile the code
C) the function signatures of the stack should not be changed
D) the component need not be changed

28) Consider the following code for deleting a node from a singly linked list. If key is present in the first node, what happens?

```
void delete(node_t **head, int key)
{
   node_t *p = *head;
   node_t *q = NULL;
   while (p != NULL && p->data != key) {
      q = p;
      p = p->next;
   }
   if (p == NULL)
      return;
   q->next = p->next;
   free(p);
}
```

A) The first node is deleted correctly and head automatically points to the second node.
B) q->next causes undefined behavior because q is NULL
C) free(p) deletes the first node, but head still safely points to the second node.
D) The function enters an infinite loop.

29) Consider the list: 10 -> 20 -> 30 -> 40 -> NULL. Where does p point after the loop?

```
struct node *p = head;
while (p->next->next != NULL)
   p = p->next;
```

A) 10

B) 20
C) 30
D) 40

30) What is the postfix expression for the following: a = b = c

A) a = b = c
B) a b c = =
C) a b = c =
D) b c = a =

Section C - L13 Skiplist / Multilist

Source: dsa unit 1,3.pdf

1) To convert from infix to postfix, we require a stack of _____ and evaluate an expression in postfix, we require a stack of _____.

A) operator, operand
B) operand, operator
C) operator, operator
D) operand, operand

2) Which of the following sequences of stack operations produces the output sequence 3, 2, 1 when the input sequence is 1, 2, 3?

A) Push 1, Push 2, Push 3, Pop, Pop, Pop
B) Push 1, Push 2, Pop, Push 3, Pop, Pop
C) Push 1, Pop, Push 2, Pop, Push 3, Pop
D) Push 1, Push 3, Pop, Push 2, Pop, Pop

3) In a skiplist, what is the main advantage of having multiple levels?

A) Faster search operations
B) Reduced memory usage
C) Easier deletion
D) Randomized balancing

4) An empty stack receives the input sequence 1, 2, 3, 4. Which of the following cannot be produced as the output sequence using only Push and Pop operations?

A) 4, 3, 2, 1
B) 2, 1, 4, 3
C) 2, 3, 4, 1
D) 3, 1, 4, 2

5) Consider the expression: ((A+B) * (C-D)) / (E-(F+G)). If the expression is scanned left to right, what is the maximum number of opening parentheses simultaneously present in the stack?

A) 2
B) 3
C) 4
D) 5

6) A multilist is best described as:

A) A linked list containing only one pointer per node
B) A linked structure in which a node can belong to more than one list
C) A circular linked list with multiple heads
D) A doubly linked list with a sentinel node

7) struct student { char name[20]; struct student* sectionNext; struct student* electiveNext; }; What is the advantage of this design?

A) Allows traversal by multiple criteria (section/elective)
B) Saves memory compared to single list
C) Prevents duplicate entries automatically
D) Eliminates need for pointers

8) An array of size 10 is used to implement two stacks growing from opposite ends. Initially top1 = -1 and top2 = 10. When is the array full?

A) top1 == top2
B) top1 + 1 == top2
C) top1 == 10
D) top2 == -1

9) A node in a multilist can contain:

A) Only one pointer
B) Multiple pointers
C) No pointer
D) Only an array index

10) Suppose a sparse matrix is represented using a multilist. To visit all non-zero elements in a particular column, which pointer should be followed repeatedly?

A) Right pointer
B) Previous pointer
C) Down pointer
D) Header pointer

11) op1, op2 are two operators. What is the postfix expression for the infix expression given below: a op1 b op2 c

A) a b c op2 op1 if op1 and op2 have the same precedence and are right associative
B) a b c op2 op1 if op1 and op2 have the same precedence and are left associative
C) a b op1 c op2 if op1 and op2 have the same precedence and are right associative
D) a b op2 c op1 if op1 and op2 have the same precedence and are left associative

12) While scanning: { A [B (C)] D (E [F]) } Immediately after reading D, what is the exact content of the stack from bottom -> top?

A) {[
B) {[(
C) {([
D) {

13) In matching of parentheses, if the stack is not empty when the input expression is completely processed, it implies

A) there are more left parentheses than the right
B) there are more right parentheses than the left
C) anyway, parentheses are well balanced
D) there can be no proper conclusion

14) Convert the following infix expression to postfix: A + B * (C ^ D - E) ^ (F + G * H) - I. Assume ^ has the highest precedence and is right-associative.

A) ABCD^E-FGH*+^*+I-
B) ABCD^E-FGH*+^*I-+
C) AB*CD^E-FG+H*^+I-
D) ABCD^E-FGH*+^+*I-

15) At the end of processing of the expression,

A) stack should have one and only one value
B) stack should be empty
C) stack may have more than one value; value at the top of the stack is the result
D) stack may have more than value; value at the bottom of the stack is the result

16) Which of the following is true of a skiplist?

A) traversal - visiting all the elements - becomes faster
B) elements should be arranged in order
C) adding elements affects only the lower level
D) Each level supports random access

17) Input sequence: 1, 2, 3, 4, 5 is pushed into a stack in this order. Which output sequence cannot be generated using only stack operations?

A) 2 1 4 3 5
B) 3 2 1 5 4
C) 3 1 2 5 4
D) 4 3 5 2 1

18) Assume the stack must match (), [], and {} correctly. Which expression contains the first mismatch when scanned from left to right?

A) {[()]}[({})]
B) [{()}]({[]})
C) {[(])}
D) ([{}]){[()]}

19) Consider the postfix expression: 5 6 2 + * 12 4 / - What is its value?

A) 37
B) 25
C) 45
D) 55

20) Convert: which infix expression will result in this postfix expression? AB+C*

A) A+B*C
B) (A+B)*C
C) A+(B*C)
D) AB+(C*)

21) What happens when a closing parenthesis) is encountered during infix-to-postfix conversion?

A) Operators are popped to output until a matching (is found and discarded
B) The) itself is pushed onto the stack
C) All operands are removed from the output
D) Nothing happens until the end of the expression

22) In the array implementation of a Stack ADT, the structure contains a char array and an integer top_. If the init_stack function initializes top_ to -1, what does top_ represent during subsequent stack operations?

A) The number of elements currently in the stack.
B) The index of the position where the next element can be added.
C) The index of the last element added (the current top element).
D) The maximum capacity of the stack array.

23) In the algorithm for matching brackets of any kind, what happens if we use only one marker to push into the stack, no matter which type of bracket is encountered?

A) ()[]{} will not be considered well formed
B) ([{}]) will not be considered well formed
C) ([)] will be considered well formed
D) none of these

24) Which of the following is the defining characteristic of a multi-list data structure?

A) Each node belongs to exactly one logical list, but the list contains multiple data types.
B) Nodes can participate in multiple logical sequences simultaneously by containing multiple pointers
C) It is simply a two-dimensional array of standard linked lists.
D) It restricts traversal to only one direction across multiple arrays.

25) A standard infix-to-postfix converter treats exponentiation (^) as right-associative, but a buggy converter treats it as left-associative when deciding whether to pop operators of equal precedence. If we evaluate the expression 2 ^ 3 ^ 2 * 2 + 1: What is the output value of the buggy evaluation? What is the output value of the correct evaluation?

A) Buggy evaluation = 65; Correct evaluation = 513
B) Buggy evaluation = 1025; Correct evaluation = 129
C) Buggy evaluation = 129; Correct evaluation = 1025
D) Buggy evaluation = 257; Correct evaluation = 1025

26) What causes a stack-based checker to immediately report an error before reaching the end of the string?

A) The stack has more than 5 elements
B) An opening bracket is pushed onto the stack
C) A closing bracket appears when the stack is empty
D) Two opening brackets appear in a row

27) In a stack-based parentheses matching algorithm, which of the following expressions would cause pop on an empty stack?

A) (A+B)
B) ((A+B))
C) (A+B))
D) ((A+B)

28) Which data structure is used to convert an infix expression to postfix?

A) stack of operands
B) stack of operators
C) queue of operands
D) queue of operators

29) Convert the given infix to postfix: 2 ^ 3 ^ 2 * 2 + 1

A) 2 3 2 ^ ^ 2 * 1 +
B) 2 3 ^ 2 ^ 2 * 1 +
C) 2 3 2 2 ^ ^ * 1 +
D) 2 3 2 2 1 ^ ^ * +

30) During a search in a skip list, when the current node's next node contains a key greater than the search key, the algorithm generally:

A) Moves up one level.
B) Moves backward.
C) Moves down one level.
D) Restarts from the head.

31) A stack is used to validate: {[(A+B) * (C-D)] + E}. During processing, which condition guarantees that the expression is syntactically balanced?

A) Stack must never become empty before the final symbol.
B) Every closing delimiter must match the stack top, and the stack must be empty after processing all symbols.

C) Number of opening delimiters must equal number of closing delimiters.
D) Every opening delimiter must be followed immediately by its corresponding closing delimiter.

32) When checking (), [], and {}, what should be pushed onto the stack when an opening bracket is encountered?

A) Always the parenthesis (
B) Always the same marker
C) different markers, one for each type of bracket
D) nothing should be pushed

33) In the middle of processing of an expression with brackets, what does stack empty imply? What does stack has elements imply?

A) expression balanced so far; expression has more left brackets than the right
B) expression is not well balanced; expression has more left brackets than the right
C) expression balanced so far; expression has more right brackets than the left
D) expression is not well balanced; expression has more right brackets than the left

34) Which of the following is not a good candidate for multilist?

A) list students based on sections or based on elective selected
B) list of words and for each word, the page number on which it appears in a book (index)
C) list of cities and for each city, a list of places to visit
D) list of subjects a student has taken in an exam and the grades in each subject

35) Corrupted expression: 5 1 2 + 4 * + 3. What's the final stack state, and what does it indicate?

A) [17, 3]
B) [17] valid
C) Underflow at +
D) [14,3,5]

36) Evaluate 6 3 2 4 + - *.

A) -18
B) 18
C) -6
D) Stack underflow

37) When an operator is applied during postfix evaluation, what's the correct operand order?

A) Operands aren't used in any specific order
B) Order doesn't matter
C) The first-popped operand always goes first
D) The second-popped operand goes first

38) A stack is used to check whether the brackets in an expression are balanced. After scanning the entire expression, the algorithm finds that the stack is not empty. What does this indicate?

A) The expression contains extra closing brackets.
B) The expression contains one or more opening brackets that do not have matching closing brackets.
C) The expression is balanced, but the stack was not cleared.
D) The expression contains invalid operators.

39) The major advantage of a multilist representation is:

A) It eliminates pointers
B) It allows multiple ways of traversing/accessing the same data
C) It always requires less memory than arrays

D) It sorts the data automatically

40) During stack-based parentheses checking, at what point can the expression be identified as invalid?

A) When the stack first becomes empty
B) When a) is encountered while the stack is empty
C) Only after all characters are processed
D) none of these

ANSWER KEY - END ONLY

Answers are given only in this final section. Q6 in Section B is technically undefined for odd-length lists; the source options do not state this exactly, so D is used as the best technical answer.

Section A - L4 Revision (C Fundamentals)

Q1	B	**Q2**	B
Q3	B	**Q4**	A
Q5	A	**Q6**	B
Q7	C	**Q8**	A
Q9	A	**Q10**	D
Q11	B	**Q12**	B
Q13	B	**Q14**	A
Q15	B	**Q16**	C
Q17	B	**Q18**	C
Q19	B	**Q20**	C
Q21	B	**Q22**	C
Q23	B	**Q24**	C
Q25	B	**Q26**	B
Q27	A	**Q28**	C
Q29	C	**Q30**	B

Section B - L8 Stack / List

Q1	C	**Q2**	A
Q3	C	**Q4**	C
Q5	C	**Q6**	D
Q7	A	**Q8**	B
Q9	D	**Q10**	B
Q11	C	**Q12**	B
Q13	A	**Q14**	B
Q15	B	**Q16**	A
Q17	B	**Q18**	D
Q19	C	**Q20**	C
Q21	B	**Q22**	A
Q23	B	**Q24**	C
Q25	B	**Q26**	A
Q27	A	**Q28**	B
Q29	C	**Q30**	B

Section C - L13 Skiplist / Multilist

Q1	A	**Q2**	A
Q3	A	**Q4**	D
Q5	B	**Q6**	B
Q7	A	**Q8**	B
Q9	B	**Q10**	C
Q11	A	**Q12**	A
Q13	A	**Q14**	A
Q15	A	**Q16**	B
Q17	C	**Q18**	C
Q19	A	**Q20**	B
Q21	A	**Q22**	C
Q23	C	**Q24**	B
Q25	C	**Q26**	C
Q27	C	**Q28**	B
Q29	A	**Q30**	C
Q31	B	**Q32**	C

Q33	A	**Q34**	B
Q35	A	**Q36**	A
Q37	D	**Q38**	B
Q39	B	**Q40**	B